

Ciencias de la Computación II

Informe de Actividad Realizada en Clase

Tema: Algoritmos de Caminos Cortos

Profesor: Simar Herrera

Integrantes:

Sebastián Rangel - 20231020006

Miguel Bueno - 20251020093

Sebastián Zambrano - 20251020102

Diego Yañes - 20251020103

Intersemestrales Junio - Julio del 2026

Universidad Distrital Francisco José de Caldas

LINK REPOSITORIO DE GIT-HUB:

<https://github.com/SBSTN18/CienciasII/tree/main>

Introducción:

Los algoritmos de caminos más cortos constituyen una de las áreas más importantes dentro de la teoría de grafos y la informática, ya que permiten determinar la ruta de menor costo entre uno o varios vértices de una red. Estos algoritmos tienen aplicaciones en numerosos campos, como sistemas de navegación GPS, redes de telecomunicaciones, videojuegos, logística, transporte, inteligencia artificial y diseño de redes de computadores.

Durante esta práctica se realizó el estudio e implementación de tres de los algoritmos más representativos para la búsqueda de caminos mínimos: Dijkstra, Bellman-Ford y Floyd-Warshall. Cada uno presenta características particulares, ventajas y limitaciones dependiendo del tipo de grafo y de las condiciones del problema que se desea resolver.

Objetivo de la Actividad:

Analizar e implementar los algoritmos de caminos más cortos Dijkstra, Bellman-Ford y Floyd-Warshall, evaluando su eficiencia, complejidad computacional y los diferentes escenarios en los que cada uno ofrece un mejor desempeño.

Algoritmos:

Algoritmo de Dijkstra

El algoritmo de Dijkstra fue propuesto por el científico neerlandés Edsger W. Dijkstra en 1956 y publicado oficialmente en 1959.

Es uno de los algoritmos más conocidos para resolver el problema del camino más corto desde un único vértice origen hacia todos los demás vértices de un grafo cuyos pesos sean positivos.

Su funcionamiento se basa en una estrategia voraz (Greedy), seleccionando en cada paso el vértice con la menor distancia conocida y actualizando posteriormente las distancias de sus vecinos.

Funcionamiento

1. Inicializa todas las distancias con infinito.
2. La distancia del nodo origen se establece en cero.
3. Se selecciona el nodo no visitado con menor distancia.
4. Se actualizan las distancias de todos sus vecinos.
5. El nodo se marca como visitado.
6. El proceso continúa hasta visitar todos los vértices.

Una vez finalizado, se obtiene la distancia mínima desde el origen hacia todos los demás nodos.

Ventajas

- Muy rápido en grafos con pesos positivos.
- Fácil de implementar.
- Excelente rendimiento para grafos grandes y dispersos.
- Muy utilizado en aplicaciones reales.

Desventajas

- No funciona correctamente cuando existen pesos negativos.
- No detecta ciclos negativos.

Algoritmo de Bellman-Ford

El algoritmo Bellman-Ford fue desarrollado por Richard Bellman y Lester Ford Jr. para resolver el problema del camino más corto desde un único nodo origen hacia todos los demás vértices, incluso cuando existen aristas con pesos negativos.

A diferencia de Dijkstra, Bellman-Ford no utiliza una estrategia voraz, sino que realiza múltiples relajaciones sobre todas las aristas del grafo hasta garantizar que todas las distancias mínimas han sido encontradas.

Además, este algoritmo posee una característica muy importante: puede detectar ciclos de peso negativo, lo cual resulta fundamental en determinadas aplicaciones.

Funcionamiento

1. Inicializa todas las distancias con infinito.
2. La distancia del nodo origen se establece en cero.
3. Recorre todas las aristas del grafo.
4. Relaja cada una de las aristas.
5. Repite el proceso $V-1$ veces.
6. Finalmente verifica si existe algún ciclo negativo.

Ventajas

- Soporta pesos negativos.
- Detecta ciclos negativos.
- Algoritmo sencillo conceptualmente.
- Muy útil para redes financieras.

Desventajas

- Más lento que Dijkstra.
- Recorre muchas veces las mismas aristas.
- Poco eficiente en grafos muy grandes.

Algoritmo de Floyd-Warshall

El algoritmo de Floyd-Warshall fue desarrollado por Robert Floyd y Stephen Warshall y pertenece a la familia de algoritmos basados en programación dinámica.

Su principal característica es que calcula simultáneamente el camino más corto entre todos los pares de vértices del grafo, en lugar de hacerlo desde un único origen.

Para ello utiliza una matriz de distancias que se actualiza progresivamente considerando cada vértice como posible punto intermedio entre dos nodos.

Funcionamiento

1. Construye una matriz de distancias inicial.
2. Considera cada vértice como un posible nodo intermedio.
3. Compara si pasar por dicho nodo reduce la distancia existente.
4. Actualiza la matriz cuando encuentra un camino más corto.
5. Repite el proceso para todos los vértices.

Al finalizar se obtiene la distancia mínima entre cualquier par de nodos del grafo.

Ventajas

- Calcula todos los caminos mínimos simultáneamente.
- Implementación sencilla.
- Excelente para grafos pequeños o medianos.
- Admite pesos negativos (sin ciclos negativos).

Desventajas

- Alto consumo de memoria.
- Poco eficiente en grafos muy grandes.
- Mayor tiempo de ejecución que Dijkstra para búsquedas individuales.

Comparación del Funcionamiento Interno

Aunque los tres algoritmos resuelven el problema del camino más corto, utilizan estrategias completamente diferentes.

- Dijkstra emplea una estrategia voraz (Greedy), seleccionando en cada paso el vértice más cercano al origen y actualizando únicamente sus vecinos.
- Bellman-Ford utiliza un proceso de relajación repetitiva sobre todas las aristas del grafo, lo que le permite trabajar con pesos negativos, aunque a costa de un mayor tiempo de ejecución.
- Floyd-Warshall implementa programación dinámica, calculando simultáneamente la distancia mínima entre todos los pares de vértices mediante una matriz que se actualiza progresivamente considerando cada nodo como posible intermediario.

Cuadro Comparativo

Característica	Dijkstra	Bellman-Ford	Floyd-Warshall
Año de creación	1959	1958	1962
Diseñador	Edsger Dijkstra	Richard Bellman y Lester Ford Jr.	Robert Floyd y Stephen Warshall
Tipo de algoritmo	Voraz (Greedy)	Relajación sucesiva	Programación dinámica
Pesos negativos	No	Sí	Sí (sin ciclos negativos)
Complejidad temporal	$O(V^2)$	$O(V \cdot E)$	$O(V^3)$
Complejidad espacial	$O(V+E)$	$O(V)$	$O(V^2)$

Característica	Dijkstra	Bellman-Ford	Floyd-Warshall
Velocidad de ejecución	Muy alta	Media	Baja
Implementación	Media	Fácil	Fácil
Adecuado para grafos grandes	Sí	Sí, aunque más lento	No es recomendable

Conclusiones

- El algoritmo de Dijkstra demostró ser el más eficiente para grafos con pesos positivos, ofreciendo tiempos de ejecución reducidos y un bajo consumo de recursos computacionales.
- Bellman-Ford presentó un mayor costo computacional debido a las múltiples relajaciones realizadas sobre las aristas, pero su capacidad para trabajar con pesos negativos y detectar ciclos negativos lo convierte en una herramienta fundamental para determinados problemas.
- Floyd-Warshall destacó por calcular simultáneamente los caminos mínimos entre todos los pares de vértices mediante programación dinámica, siendo especialmente útil en problemas de conectividad global.